

Introduction à Git

4 heures sur les bases: 2 séances de 2h ;

- 2 heures jusqu'à la partie *Gérer les conflits*.
- 2 heures sur l'historique, les branches, les conflits et la collaboration.
- **Pas d'évaluation.**

Cours de Git de M. Pierre-Antoine Champin : <https://vvidal.github.io/cours-git-intro/cours-git-intro/>

Mots clefs: dépôt (local+distant), commit, branch, merge, pull, push, résolution de conflits

Objectifs du cours:

- Comprendre les avantages d'un système de control des versions ;
- Apprendre à utiliser Git dans un projet collaboratif (**niveau débutant**)
 - Comment « versionner » un projet ?
 - Les lignes de commandes de base et l'utilisation d'une IHM (TortoiseGit et/ou GitKraken)
 - Pour travailler sur son dépôt local ;
 - Pour synchroniser son travail avec un dépôt distant.
 - Comprendre le workflow Git.

1. Pourquoi utiliser Git?

- a. Très populaire
 - i. Utilisé par les plus grosses entreprises ;
 - ii. Ecrit en C par les développeurs du noyau Linux (très rapide).
 - iii. **Gestionnaire de version décentralisé**
 1. Pas besoin d'un serveur central, les utilisateurs peuvent se synchroniser entre eux.
 2. Mais on peut quand même utiliser un serveur central pour diffuser une version stable, permettant aux utilisateurs de se synchroniser.
- b. **Outil de travail collaboratif** (> 1 développeur)
 - i. Faciliter la collaboration, le partage de travail, et surtout le travail sur un même projet
 1. VCS (*Version Control System*) : **Les humains ne sont pas doués pour faire de la gestion de version** (de fichiers et de dossiers) alors que les logiciels dédiés le sont (moins coûteux en place disque) ;
 - a. Nommage des versions, conservation de l'ordre des versions et gestion des conflits.
 2. Echanges sécurisés par SSH.
 - ii. Pour quels documents ? Au départ surtout pour du code, mais aussi de la documentation (surtout des fichiers textes non-formatés) et des sites Web ;
 - iii. Nominatif : Qui a fait quoi ? (Qui a introduit le bogue ? Qui doit être payé ?)
 - iv. Conservation d'un historique : ne pas perdre le passé : quand cela a été fait ? Rejouer le passé (défaire un travail pour toujours).
 - v. Fusion des modifications et gestion des conflits.
- c. **Avantages de la gestion de version**
 - i. Sauvegarde modulo une synchronisation avec un serveur distant ;
 - ii. Visualiser les changements au cours du temps ;

- iii. Conservation d'un historique et fusion des modifications;
- iv. Avoir plusieurs versions pour résoudre un même problème qui coexistent ; on peut les tester en parallèle pour au final ne retenir que la meilleure.
- d. Des serveurs Git de qualité (et avec des fonctionnalités intégrées)
 - i. GitHub : <https://github.com/>
 - 1. Attention à utiliser un projet privé pour vos travaux à l'IUT
 - ii. GitLab : <https://about.gitlab.com/>
 - 1. Serveur GitLab de l'université Lyon 1 : <https://forge.univ-lyon1.fr>
 - a. Login et mdp de l'IUT.
 - iii. Bitbucket: <https://bitbucket.org/>
 - iv. Framagit: <https://framagit.org/>

2. Les notions de base

- a. **Dépôt** (*repository*)
 - i. Le répertoire caché .git ;
 - ii. Contient toutes les données dont Git a besoin pour gérer l'historique.
- b. **Commit**
 - i. Si l'historique d'un projet est une séquence de photos/instantanés, le commit représente une telle photo = **un ensemble de modifications et de fichiers ajoutés ou supprimés que l'on souhaite conserver et identifier** (via la référence du commit) **depuis le dernier commit** ;
 - ii. Contenu d'un commit : Date+auteur+description/message+liens vers d'autres commits ;
 - iii. Ce qui est stocké lors d'un commit ? Pour chaque commit, uniquement les différences sont stockées (et non pas la totalité des fichiers) ;
 - iv. Quand doit-on commiter ? Faire un commit après un ensemble cohérent de modifications ;
 - v. Je peux éditer ou regrouper mes commits **avant** de les publier.
- c. **Copie de travail** (*working copy*)
 - i. Les fichiers locaux présents dans le répertoire géré par GIT (ces fichiers peuvent être différents du dernier commit).
- d. **Index** (*staging area*)
 - i. **Espace temporaire** (fichier) contenant les informations prêtes à être enregistrées avec la commande `git commit`
 - 1. Modifications, suppression et ajout de fichiers.
 - ii. Remarque : uniquement les fichiers ajoutés à la **staging area** seront pris en compte lors du prochain commit (ainsi un fichier créé mais non ajouté à la staging area ne sera pas enregistré dans l'historique lors du prochain commit).

3. Les outils de mise en œuvre

- a. Git Gui (interface graphique) et **Git Bash** (interpréteur de lignes de commande)
 - i. <https://git-scm.com/>
- b. Autres IHM populaires (clients Git)
 - i. Recommandée : **TortoiseGit** (il faut installer Git avant) : <https://tortoisegit.org/>
 - ii. GitKraken (pas la peine d'installer Git avant) : <https://www.gitkraken.com/>
- c. Serveur GitLab de l'université Lyon 1
 - i. <https://forge.univ-lyon1.fr>
 - 1. Login et mdp de l'IUT.

4. Workflow

- a. **Installer Git** sur sa machine et avoir un compte sur un serveur Git ;
 - i. <http://git-scm.com/downloads>
 - ii. Optionnel : installer TortoiseGit après Git : <https://tortoisegit.org/>
- b. **Configurer GIT** (spécifier votre nom d'utilisateur et votre adresse mél) ;
 - i. `git config --global user.name "NOM PRENOM"`
 - ii. `git config --global user.email "prenom.nom@etu.univ-lyon1.fr"`
- c. **Avoir un dépôt Git local**
 - i. `git clone` (copie d'un dépôt distant) ou `git init/git init directory` (création d'un dépôt local, le dossier caché .git).
 - ii. On travaille sur sa copie locale.
- d. **Travailler sur sa copie de travail**
 - i. Modifier un fichier et/ou ajouter un fichier et/ou supprimer des fichiers dans le répertoire géré par le dépôt local ;
- e. **Ajouter certaines modifications à l'index**
 - i. `git add filename` (ajouter les modifications du fichier *filename* à l'index) ou `git reset filename` (enlever de l'index les dernières modifications ajoutées concernant le fichier *filename*)
- f. **Faire un commit** (ou enregistrer dans l'historique) l'ensemble des modifications se trouvant dans l'index ;
 1. `git commit -m "description"`
- g. **Etapes précédentes en vidéo** : <https://git-scm.com/video/get-going> ;
- h. **branching workflow** : une branche va contenir un ensemble de commits proposant une nouvelle fonctionnalité (*feature* en anglais) ou corrigeant un bogue (un commit non associé à une branche est automatiquement supprimé/perdu)
 - i. Isolation du développement de chaque feature / de chaque correctif
 1. Traiter des bogues dans une prochaine version du logiciel (*bugfix*) ;
 2. Traiter des bogues pour la version actuelle du logiciel (*hotfix*) ;
 3. Savoir si le code d'une feature particulière est complet ou pas ;
 4. Savoir si le code d'une feature particulière a été relu ;
 5. Gérer plusieurs versions d'un logiciel ;
 6. Convention de nommage des branches: branch_type/issue_key-description
 - a. featureXXX/JIRA-47-copy-constructor ;
 - b. bugfix/JIRA-83-string-encoding ;
 - c. hotfix/JIRA-56-ie8-menu-display.
 - ii. Stabilité de la branche master (branche de production)
 1. Jamais de build cassé ;
 2. Jamais de feature incomplete ;
 3. En temps normal, notre copie de travail est synchronisée avec la branche master.
 - iii. Traçabilité des features implantées
 1. Branches déjà fusionnées dans la branche actuelle: `git branch --merged`
 2. Branches pas encore fusionnées dans la branche actuelle: `git branch --no-merged`
 - iv. **On travaille sur une seule branche à la fois (à un instant précis)**
 1. `git checkout` branche pour travailler sur la branche branche ;

- a. **Avant tout changement de branche s'assurer que la branche actuelle ne contient pas de modification non-enregistrée dans l'historique.**
2. pour faire le checkout sur une branche nouvelle à créer : **git checkout -b nouvelle_branche** ;
3. A chaque nouveau commit, le sommet de la branche courante est avancé vers ce nouveau commit ;
4. Si on a une version distante (*remote*) et une version locale de sa branche
 - a. Dès qu'on arrive sur une machine où la version locale (sans modification non-présente dans l'historique) est en retard par rapport à la version en ligne => **on fait un git pull**
 - i. On fait également régulièrement un **git pull master** (+idéalement celui du dépôt central) pour ne pas avoir une solution trop divergente.
 - b. Dès qu'on a terminé de travailler en local, qu'on a fait un commit, qu'on a bien récupéré tous les commits distants, **on fait un git push** de sa branche pour copier son travail sur la branche distante.
- i. Fusionner les branches
 - i. *master* branch (production) ;
 - ii. *develop* branch (branche d'intégration, qui peut être cassée à un moment donné par une fusion) ;
 - iii. Suite à une fusion
 1. Gestion des conflits (*merge conflict*)
 - a. Même fichier binaire modifié ou même ligne modifiée.
 2. Pour chaque fichier ayant un conflit, dès que le conflit est résolu, on fait un **git add fichier_conflit_resolu** (on ajoute le fichier à l'index) ;
 3. Créer un commit (*merge commit*) si pas de fast-forward, cette création étant automatique en l'absence de conflit ;
 4. Une fois la fusion terminée, il ne faut pas oublier de supprimer la branche d'une feature ou de la correction d'un bogue. **git branch -d** branche

5. Les commandes

- a. Apprendre et pratiquer les commandes
 - i. <https://onlywei.github.io/explain-git-with-d3/#>
 - ii. <http://pcottle.github.io/learnGitBranching/>
- b. Créer un dépôt local : **git init** dans le dossier, on peut également cloner un dépôt distant existant via **git clone url_du_dépôt_distant** ;
 - i. Par défaut une seule branche *master* (ou *main*) ;
 - ii. Attention pour la commande **git clone** : Si vous travailler en *ssh* ou en *https* (à privilégier à l'IUT) l'url n'est pas la même ! Par exemple pour la forge de lyon1 :
 1. En *ssh* : **git clone git@forge.univ-lyon1.fr:votre_login/nom_projet.git**
 2. En *https* : **git clone https://forge.univ-lyon1.fr/votre_login/nom_projet.git**
 3. En cas d'erreur d'url : **git remote set-url origin votre_url**
- c. Ajouter un fichier (adding/staging) à l'index : **git add nom_fichier** ;
- d. Retirer un fichier de l'index : **git reset nom_fichier** ;
- e. Prendre en compte les modifications dans l'index au sein d'un commit : **git commit -m "message décrivant les modifications"** ;
- f. Quelle est la situation actuelle de l'index ? **git status** ;

- g. Historique : quel est le passé de mon projet ? **git log**
 - i. Ce qui a été fait depuis hier ? **git log --since "yesterday"** ;
 - ii. Quel est le passé du contributeur nom ? **git log --nom** ;
 - iii. Afficher le détail d'un commit particulier : **git show id-commit** ;
 - iv. Cette ligne a été introduite par quel commit ? **git gui blame fichier**.
- h. Travailler avec plusieurs versions du même projet en même temps : les branches
 - i. Lister les branches (locales) actuelles : **git branch** ;
 - ii. Créer une nouvelle branche : **git branch** branche_à_créer ;
 - iii. Supprimer une branche : **git branch -d** branche_à_supprimer ;
 - iv. Changer de branche : **git checkout** branche ;
 - v. Comment intégrer les modifications d'1 branche dans 1 autre ?
 - 1. **Fusionner 2 branches (privées ou publiques)** : **git merge** branche_à_merger_sur_l_actuelle pour intégrer deux historiques divergents (fusion) ou décalés linéairement dans le temps (fast-forward, uniquement le pointeur de commit est avancé)
 - a. On souhaite intégrer les corrections de la branche *correctif* dans la branche *master* : **git checkout master** (la branche de travail est la branche *master*) puis **git merge correctif** et éventuellement **git branch -d correctif** pour supprimer la branche *correctif* (dont on n'a plus besoin)
 - i. <https://git-scm.com/book/fr/v2/Les-branches-avec-Git-Branches-et-fusions%C2%A0%3A-les-bases>
 - b. Cela créera un commit de fusion.
 - 2. **Rebaser une branche (privée, i.e. non publiée)** : **git rebase** branche_sur_laquelle_rejouer_ses_commits pour rejouer ses commits à partir de l'ancêtre commun à la suite des dernier commits de l'autre branche
 - a. On se met sur la branche à « déplacer » (e.g. *feature*), puis on rebase sur une branche destination (e.g. *master*), et ensuite on se place sur la branche de destination (e.g. **git checkout master**) et finalement on fait un merge avec la branche à déplacer (e.g. **git merge feature** => fusion en avance rapide). On peut terminer par la suppression de la branche à déplacer (e.g. **git branch -d feature**).
 - i. <https://www.youtube.com/watch?v=dO9BtPDIHJ8>
 - ii. <https://git-scm.com/book/fr/v2/Les-branches-avec-Git-Rebaser-Rebasing>
 - b. Une commande similaire : **git rebase** branche_de_base branche_de_sujet .
- i. **Les branches de suivi (remotes-tracking branches)**
 - i. Informations d'un dépôt distant
 - 1. **git remote show** nom_remote
 - ii. Ajouter un remote (e.g., pour un collaborateur)
 - 1. **git remote add** nom_remote url_du_dépôt_distant .
 - iii. Supprimer un remote
 - 1. **git remote rm** nom_remote .
 - iv. Créer une branche locale de suivi basée sur une branche distante
 - 1. **git checkout -b** branche_locale nom_remote/branche_distante ;

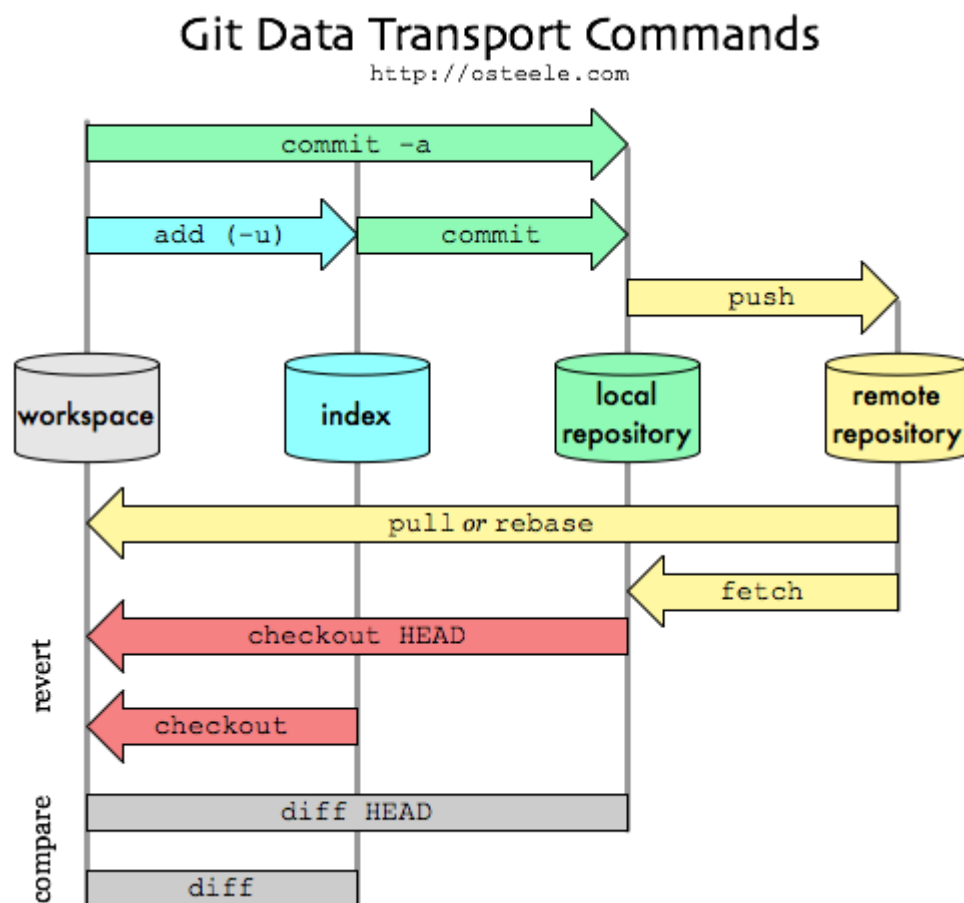
2. **git branch -vv** pour lister les branches locales et leurs branches de suivi associées
- v. Associer une nouvelle branche créée dans le dépôt local à une branche de suivi de même nom
 1. **git push --set-upstream** nom_remote branche_locale
- vi. Mettre à jour les branches de suivi locales à partir des derniers changements du remote (sans intégrer/fusionner les modifications à la branche locale actuelle)
 1. **git fetch** nom_remote branche_du_remote_en_question ;
 2. **git fetch** nom_remote .
- vii. Fusionner une branche d'un remote avec ma branche locale courante
 1. **git merge** nom_remote/branche_distante ;
 2. Attention : on ne merge qu'une version déjà fetchée
 - a. Solution pour ne plus oublier : **git pull** nom_remote branche_du_remote_en_question .
 - b. Pour préserver un ordre linéaire des commits : **git pull --rebase**
 3. Pour abandonner une fusion (e.g. à cause d'un conflit ingérable dans l'immédiat) : **git merge --abort**
- viii. Supprimer une branche sur un remote : **git push origin :branche_à_supprimer**
- ix. Différence entre git pull et git fetch ?
 1. **git pull** fait un **git fetch** suivi d'un **git merge** : **git pull** est la commande à faire pour mettre à jour sa branche locale courante avec la version en ligne sélectionnée ; attention cependant, **le git pull merge les fichiers pour vous à vos risques et périls. Il faut donc toujours vérifier le résultat !**
- x. Incorporer des changements locaux dans des branches distantes (si on a les permissions !)
 1. **git push** nom_remote branche_du_remote_en_question ;
 2. Attention : avant tout push : **git branch** pour savoir si je suis bien sur la bonne branche, puis un **git pull** de la branche distante (pour être à jour), puis éventuellement des conflits à gérer et un **git commit** pour signaler que les conflits ont été résolus et finalement je peux pusher.

6. Travailler sur le GitLab de l'université (<https://forge.univ-lyon1.fr>) en collaboration avec d'autres utilisateurs

- a. 1 étudiant crée le dépôt distant et se donne tous les droits (Maintainer ⇔ admin) ;
- b. Il invite d'autres étudiants qu'il va mettre en Developer ; Il protège sa branche master (ou main) en autorisant uniquement les Maintainers à merger ou pusher sur cette branche ;
- c. 2 manières de travailler
 - i. La manière « simple » : Sur le projet, on a la branche master (ou main) et une branche par développeur (y compris le Maintainer si ce dernier code)
 1. master (ou main)
 2. dev-simon
 3. dev-amelie
 4. etc.
 5. et on merge sa branche avec le master après chaque tâche terminée (et qui ne casse pas master).

6. Cette approche peut se faire qu'avec un seul remote d'enregistré sur son dépôt local, celui du dépôt distant géré par le Maintainer.
- ii. La manière « classique » : chaque membre invité fait un fork du projet (crée une divergence), pour avoir sa propre version du projet distant.
 1. On a donc le projet/dépôt central
 2. Le fork de Simon
 3. Le fork d'Amélie
 4. etc.
 5. Et on merge une de ses branches de son fork avec la branche master du dépôt central après chaque tâche terminée (et qui ne casse pas master).
 6. Cette approche nécessite d'avoir au moins 2 remotes d'enregistrés sur son dépôt local, celui du dépôt central (pour se mettre à jour régulièrement) et son dépôt distant (origin) pour se synchroniser régulièrement.
 7. L'étudiant qui possède le dépôt original doit rendre la branche master de son projet « protégée » (il faut aller dans les paramètres du projet) => il ne donne les droits de modification qu'aux Maintainers.
 - a. Cela forcera les étudiants invités Developer, qui veulent faire remonter leur travail dans la branche master, à faire une merge request, merge request qui sera traitée par un Maintainer.

7. Une image vaut bien des explications



8. Aller plus loin

- a. git stash

- i. Utilisez git stash avant de changer de branche si vous avez des modifications non terminées que vous souhaitez valider ultérieurement
 - 1. <https://git-scm.com/book/en/v2/Git-Tools-Stashing-and-Cleaning>
- b. Les tags (étiquettes)
 - i. Pointeurs sur un commit, ne bougent jamais.
- c. Mise en place de ses clé rsa pour faire du SSH
 - i. ssh_keygen
 - 1. Mettre la clé publique dans GitLab [sur iutdoua-git, le ssh est bloqué, il faut donc utiliser des urls en https à la place] ;
 - 2. Indiquer la clé privée à votre EDI (eclipse, NetBeans, AndroidStudio...).
- d. Git cheat sheet :
 - i. <https://gist.github.com/hofmannsven/6814451>

9. Références bibliographiques

- a. <https://git-scm.com/book/fr/v2> ou [Livre Pro Git sur amazon](#)